Laporan Tugas Besar

IF2224 Teori Bahasa Formal dan Otomata

Milestone 1

Semester II / 2025–2026

Disusun oleh:

GTFOTBFO (HBB)

Nathanael Imandatua Manurung - 13524021
Nicholas Wise Saragih Sumbayak - 13524037
Kurt Mikhael Purba - 13524065
Rava Khoman Tuah Saragih - 13524107

Laboratorium Ilmu dan Rekayasa Komputasi
Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

Daftar Isi

1 Pendahuluan **2**

2 Dasar Teori **2**
2.1 Pendefinisian Bahasa Pemrograman . . . 2
2.1.1 Hierarki Bahasa Pemrograman . . . 3
2.1.2 *Compiler* dan *Interpreter* . . . 3
2.2 Teori Kompilator (*Compiler Theory*) . . . 3
2.2.1 Analisis Leksikal (*Lexical Analysis*) . . . 4
2.3 Teori Bahasa Formal dan Otomata . . . 5
2.3.1 *Regular Expression* . . . 5
2.3.2 *Finite Automata* (FA) . . . 5

3 Perancangan & Implementasi **6**
3.1 Diagram Transisi DFA . . . 6
3.2 Implementasi Program (Lexical Analyzer) . . . 9
3.2.1 Struktur Proyek . . . 9
3.2.2 Klasifikasi Implementasi Alur DFA dalam Program . . . 9

4 Experiments and Results **11**
4.1 Test Case Category A . . . 11
4.1.1 Test Case 1: *[Description]* . . . 11
4.1.2 Test Case 2: *[Description]* . . . 11
4.2 Test Case Category B . . . 11
4.3 Overall Analysis . . . 11
4.3.1 Algorithm A — Strengths and Limitations . . . 11
4.3.2 Algorithm B — Strengths and Limitations . . . 11

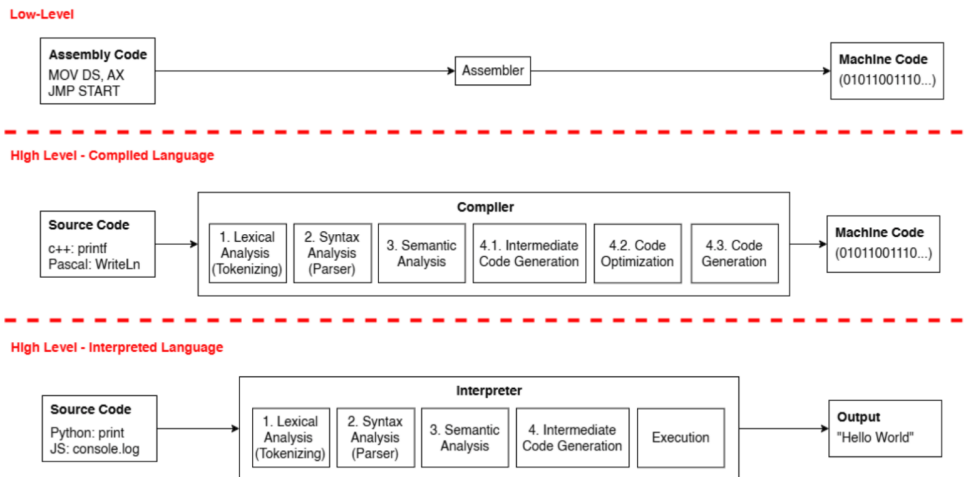
5 Conclusion **12**
5.1 Summary . . . 12
5.2 Suggestions for Future Work . . . 12
5.3 Individual Reflections . . . 12
5.3.1 Nathanael Imandatua Manurung - 13524021 . . . 12
5.3.2 Nicholas Wise Saragih Sumbayak - 13524037 . . . 12
5.3.3 Kurt Mikhael Purba - 13524065 . . . 12

References **13**

Appendix **14**

1 Pendahuluan

Bahasa Pemrograman diciptakan sebagai perantara komunikasi antara manusia yang mengerti logika dan bahasa tingkat tinggi dengan komputer yang hanya dapat mengerti bahasa mesin (*machine code*). Tanpa adanya bahasa pemrograman, manusia harus menuliskan instruksi komputer secara tingkat rendah kepada komputer yang hanya mengerti 0 dan 1. Dalam hierarkinya, bahasa tingkat tinggi (*high-level*) dirancang agar lebih mendekati bahasa manusia, namun hal ini menuntut proses transformasi yang berlapis agar dapat dipahami oleh CPU.



Gambar 1: Tipe-tipe bahasa pemrograman

Sumber: Asisten IF2224

Proses transformasi ini dilakukan melalui mekanisme *compiler* atau *interpreter*. Tahap awal dan fundamental dalam mekanisme tersebut adalah Analisis Leksikal (*Lexical Analysis*). Pada tahap ini, kode sumber yang berupa rangkaian karakter mentah dibaca dan dikelompokkan menjadi satuan makna terkecil yang disebut token, seperti kata kunci (*keyword*), pengenal (*identifier*), operator, dan konstanta. Analisis leksikal berperan krusial dalam menyederhanakan tugas tahap berikutnya, yaitu parsing, dengan membuang elemen yang tidak relevan seperti spasi dan komentar, serta mendeteksi simbol invalid sejak awal.

2 Dasar Teori

2.1 Pendefinisian Bahasa Pemrograman

Bahasa Pemrograman adalah cara manusia untuk dapat memberikan perintah (*instruction*) kepada sebuah komputer. Setiap bahasa pemrograman memiliki kata kunci dan aturannya masing-masing serta makna dari kata-kata itu yang disebut sintaks(*syntax*) dan semantik (*semantics*). Input yang terdapat kesalahan pada tingkat sintaks berarti input tersebut tidak valid untuk bahasa pemrograman tersebut, sedangkan kesalahan pada tingkat semantik umumnya berbentuk kesalahan logika.

2.1.1 Hierarki Bahasa Pemrograman

Berdasarkan tingkat abstraksinya, bahasa pemrograman diklasifikasikan menjadi dua kategori utama:

- ***Low-Level Language* (Bahasa Tingkat Rendah)**: Bahasa yang instruksinya mendekati arsitektur perangkat keras dan bahasa mesin (biner 0 dan 1). Contohnya adalah *Assembly*, yang memerlukan *Assembler* untuk menerjemahkan kode menjadi instruksi yang dapat dipahami CPU.
- ***High-Level Language* (Bahasa Tingkat Tinggi)**: Bahasa yang dirancang agar lebih mudah dibaca dan dipahami oleh manusia karena menggunakan kosakata yang menyerupai bahasa natural. Semakin tinggi tingkat bahasa tersebut, semakin banyak tahapan pemrosesan yang diperlukan untuk mengubahnya menjadi kode mesin. Contoh bahasa tingkat tinggi antara lain Pascal, C++, Python, dan Java.

2.1.2 *Compiler* dan *Interpreter*

Untuk menjalankan kode sumber yang ditulis dalam bahasa tingkat tinggi, diperlukan sebuah perangkat lunak penerjemah yang mengubah kode tersebut menjadi format yang dapat dieksekusi oleh mesin. Secara umum, terdapat dua mekanisme utama untuk mengubah bahasa tingkat tinggi ke bahasa mesin:

1. ***Compiler* (Kompilator)**: *Compiler* adalah program yang menerjemahkan seluruh *source code* secara utuh menjadi kode mesin (biasanya dalam bentuk *Object File*) sebelum program dijalankan. Proses kompilasi pada *compiler* umumnya terdiri dari empat tahapan utama: *Lexical Analysis*, *Syntax Analysis*, *Semantic Analysis*, dan *Intermediate Code Generation*. Karakteristik utama *compiler* adalah jika terdeteksi adanya kesalahan (*error*) di dalam *source code*, maka seluruh proses kompilasi akan gagal dan program tidak dapat dijalankan.
2. ***Interpreter***: Berbeda dengan *compiler*, *interpreter* memproses kode sumber baris demi baris secara langsung. *Interpreter* membaca, menerjemahkan, dan langsung mengeksekusi kode mesin tersebut baris demi baris. Hal ini memungkinkan program tetap berjalan normal hingga baris sebelum ditemukannya kesalahan; misalnya, jika terdapat *error* pada baris ke-10, baris 1 sampai 9 akan tetap berhasil dieksekusi. Meskipun mekanisme eksekusinya berbeda, *interpreter* sebenarnya memiliki tahapan pemrosesan internal yang serupa dengan *compiler*.

2.2 Teori Kompilator (*Compiler Theory*)

Proses kompilasi adalah urutan dari berbagai fase (*phases*) yang bekerja secara berurutan. Setiap fase mengambil input dari tahap sebelumnya, memprosesnya, dan memberikan output yang akan digunakan sebagai input untuk fase berikutnya. Secara umum, arsitektur sebuah kompilator terdiri dari fase-fase berikut:

1. ***Lexical Analysis***: Fase pertama yang bekerja sebagai pemindai teks (*text scanner*) pada kode sumber. Penjelasan lebih detail mengenai fase ini dibahas pada sub-bab tersendiri di bawah.

2. ***Syntax Analysis* (*Parsing*)**: Mengambil *token* yang dihasilkan oleh *lexical analysis* dan membangun pohon sintaksis (*parse tree*). Fase ini memeriksa kebenaran susunan *token* berdasarkan aturan tata bahasa (*grammar*) kode sumber.

3. ***Semantic Analysis***: Memeriksa apakah pohon sintaksis yang dibangun mengikuti aturan semantik bahasa, seperti kecocokan operasi tipe data dan memastikan *identifier* telah dideklarasikan sebelum digunakan. Fase ini menghasilkan keluaran berupa pohon sintaksis yang dianotasi.

4. ***Intermediate Code Generation***: Menghasilkan representasi kode perantara untuk mesin abstrak, yang posisinya menjembatani antara bahasa tingkat tinggi dan bahasa mesin.

5. ***Code Optimization***: Mengoptimalkan kode perantara dengan menghapus baris kode yang tidak perlu dan mengatur ulang urutan instruksi. Tujuannya adalah untuk mempercepat eksekusi program tanpa membuang sumber daya (*CPU*, memori).

6. ***Target Code Generation***: Merupakan fase sintesis akhir (*back-end*) yang menggunakan representasi kode perantara dan tabel simbol untuk menghasilkan program target atau bahasa mesin.

2.2.1 Analisis Leksikal (*Lexical Analysis*)

Sebagai fokus utama pada *milestone* ini, Analisis Leksikal adalah fase pertama dari sebuah kompilator. Fase ini memproses kode sumber yang ditulis dalam bentuk kalimat sintaksis dan memecahnya menjadi serangkaian unit bermakna yang disebut *token*.

Berdasarkan prinsip desain kompilator, mekanisme kerja dan tugas utama dari *lexical analyzer* meliputi:

- **Penyaringan Karakter**: *Lexical analyzer* membaca kode sumber sebagai aliran karakter (*character stream*) dan secara otomatis menghapus seluruh spasi kosong (*whitespace*) maupun komentar (*comments*) di dalam kode sumber.

- **Tokenisasi dan Pengenalan *Lexeme***: Barisan karakter (*alphanumeric*) yang memiliki makna diidentifikasi sebagai *lexeme*. *Lexeme* ini kemudian dicocokkan dengan aturan baku atau pola (*pattern*)—yang umumnya didefinisikan melalui *regular expression*—untuk divalidasi sebagai sebuah *token* (seperti kata kunci, konstanta, *identifier*, atau operator).

- **Kerja Sama dengan *Syntax Analyzer* (*Parser*)**: *Lexical analyzer* bekerja sangat erat dengan *syntax analyzer*. Ia memeriksa keberadaan *token* yang legal dan meneruskan data *token* tersebut kepada *syntax analyzer* hanya pada saat dibutuhkan atau diminta (*when it demands*).

- **Penanganan Kesalahan (*Error Handling*)**: Apabila *lexical analyzer* memindai urutan karakter yang tidak valid atau tidak dapat diklasifikasikan ke dalam pola *token* apa pun, maka ia akan langsung menghasilkan pesan kesalahan (*error*).

2.3 Teori Bahasa Formal dan Otomata

2.3.1 *Regular Expression*

Regular Expression (RE) adalah sebuah notasi matematis formal yang digunakan untuk mendeskripsikan suatu bahasa reguler. Dalam kompilator, RE digunakan secara luas untuk mendefinisikan *pattern* (pola) dari *token* secara akurat. Dengan menggunakan operasi dasar seperti *Concatenation* (penyambungan), *Union* (penggabungan alternatif), dan *Kleene Closure* (perulangan nol kali atau lebih), pembuat bahasa dapat mendefinisikan aturan rumit, misalnya: "Sebuah identifier harus diawali dengan huruf, diikuti oleh kombinasi huruf atau angka dengan jumlah berapa pun."

2.3.2 *Finite Automata* (FA)

Finite Automata (FA) atau *Finite State Machine* (FSM) adalah model komputasi matematika abstrak yang digunakan untuk mengenali *Regular Expression*. Dalam konteks kompilator, FA berfungsi sebagai mesin pemeriksa yang menelusuri karakter masukan untuk memvalidasi apakah urutan karakter tersebut (*lexeme*) membentuk *token* yang sah sesuai aturan bahasa.

Secara formal, sebuah *Finite Automata* didefinisikan secara matematis sebagai *5-tuple* $(Q, \Sigma, \delta, q_0, F)$, yang terdiri dari:

- Q : Himpunan hingga dari kedudukan atau *state*.
- Σ : Himpunan hingga dari simbol *input* atau alfabet (contoh: himpunan huruf, angka, dan simbol baca).
- δ : Fungsi transisi yang memetakan $Q \times \Sigma \rightarrow Q$ (menentukan *state* tujuan berdasarkan *state* saat ini dan simbol yang dibaca).
- q_0 : *State* awal (*initial state*), di mana $q_0 \in Q$. Mesin selalu memulai komputasi dari *state* ini.
- F : Himpunan *state* akhir atau penerima (*final/accepting state*), di mana $F \subseteq Q$.

Mesin FA membaca masukan berupa *string* karakter demi karakter secara sekuensial dari kiri ke kanan. Setiap kali membaca satu simbol, mesin akan berpindah dari *state* saat ini ke *state* berikutnya sesuai dengan aturan pada fungsi transisi (δ). Jika setelah seluruh karakter pada masukan selesai dibaca mesin berhenti pada salah satu *state* yang termasuk dalam himpunan F, maka masukan tersebut diterima (*accepted*) sebagai *token* yang valid. Sebaliknya, jika berhenti di luar F atau menemukan transisi yang tidak terdefinisi, maka terjadi *lexical error*.

Berdasarkan sifat penentuan fungsi transisinya, *Finite Automata* dibedakan menjadi dua jenis:

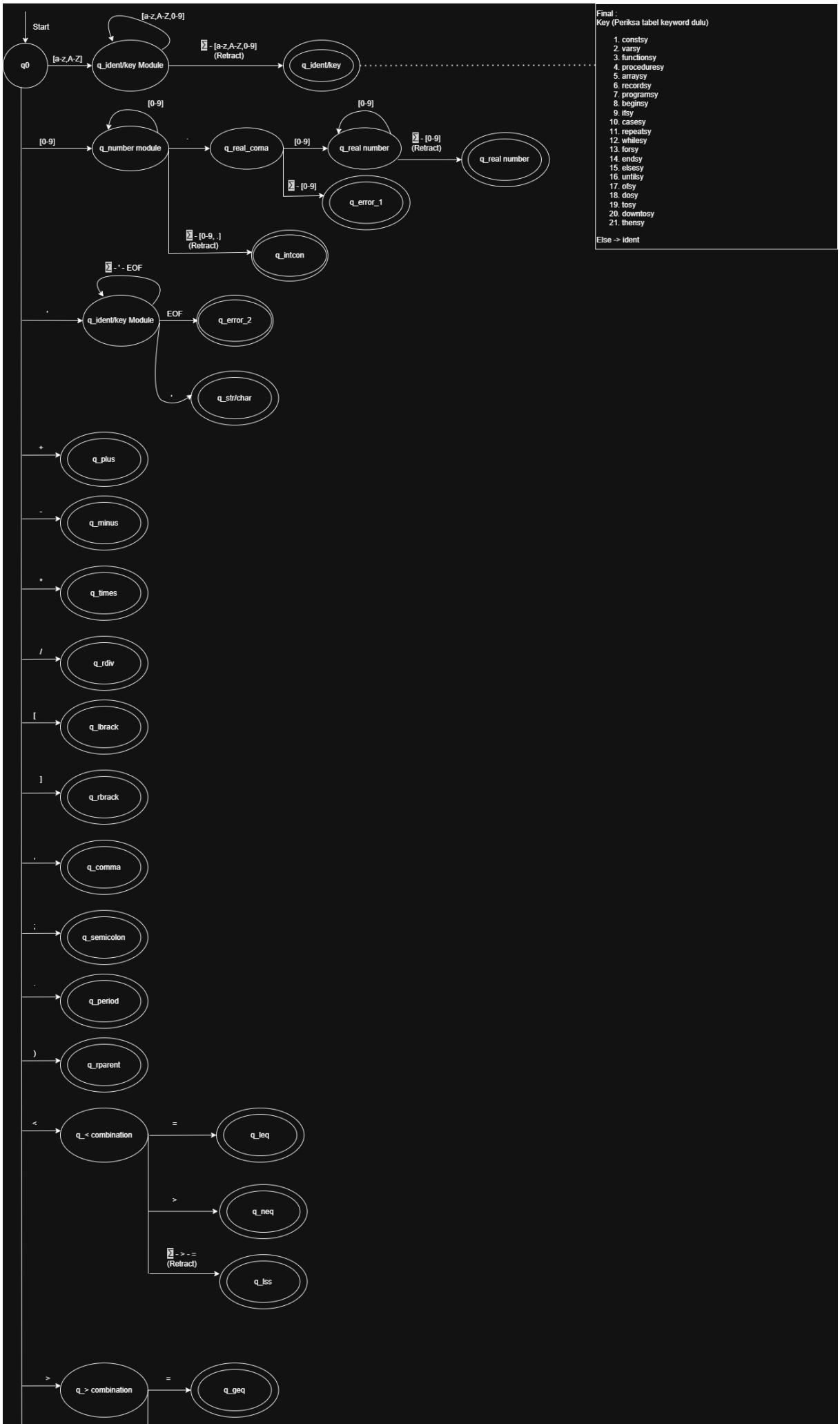
1. ***Nondeterministic Finite Automata* (NFA)**: Suatu mesin di mana sebuah *state* dapat memiliki nol, satu, atau lebih jalur transisi untuk satu simbol *input* yang sama. NFA juga mengizinkan perpindahan *state* tanpa perlu membaca karakter *input* (disebut transisi epsilon atau ϵ-transition).
2. ***Deterministic Finite Automata* (DFA)**: Suatu mesin di mana setiap *state* memiliki tepat satu jalur transisi secara mutlak untuk setiap simbol *input* di dalam Σ. Tidak ada ambiguitas jalur dan tidak ada transisi epsilon.

Dalam implementasi program *Lexical Analyzer* yang sebenarnya, model **DFA** adalah spesifikasi yang wajib digunakan. Hal ini karena sifat deterministiknya yang tidak ambigu memungkinkan aturan transisi dikonversi secara langsung menjadi algoritma terprogram menggunakan pendekatan *modular* dan penyeleksian kondisi, membaca satu demi satu karakter hingga menghasilkan klasifikasi *token* yang tepat.

3 Perancangan & Implementasi

3.1 Diagram Transisi DFA

Berikut merupakan rancangan *Deterministic Finite Automata* (DFA) yang dikembangkan untuk mengenali seluruh pembentukan leksikal token pada bahasa pemrograman Arion.


Start
q0
[a-z,A-Z]
q_ident/key Module
[a-z,A-Z,0-9]
∑ - [a-z,A-Z,0-9]
(Retract)
q_ident/key
Final :
Key (Periksa tabel keyword dulu)
1. constsy
2. varsy
3. functionsy
4. proceduresy
5. arraysy
6. recordsy
7. programsy
8. beginsy
9. ifsy
10. casesy
11. repeatsy
12. whilesy
13. forsy
14. endsy
15. elsesy
16. untilsy
17. ofsy
18. dosy
19. tosy
20. downtosy
21. thensy
Else -> ident
[0-9]
q_number module
q_real_coma
q_real number
∑ - [0-9]
(Retract)
q_real number
q_error_1
∑ - [0-9, .]
(Retract)
q_intcon
∑ - ' - EOF
q_ident/key Module
EOF
q_error_2
q_str/char
+
q_plus
-
q_minus
*
q_times
/
q_rdiv
[
q_lbrack
]
q_rbrack
,
q_comma
;
q_semicolon
.
q_period
)
q_rparent
<
q_< combination
=
q_leq
>
q_neq
∑ - > - =
(Retract)
q_lss
q_> combination
q_geq

Perancangan diagram transisi DFA ini merupakan representasi logis dari bagaimana *scanner* membaca karakter demi karakter (*character stream*) dan menentukan validitas sebuah *lexeme*. Berdasarkan diagram di atas, terdapat beberapa keputusan desain dan analisis perancangan *state* yang diterapkan:

1. Mekanisme *Look-ahead* pada Operator Relasional dan *Assignment*

Bahasa Arion memiliki operator yang terdiri dari satu karakter (seperti `+`, `-`, `=`) dan multi-karakter (seperti `<=`, `>=`, `<>`, `:=`). Pada DFA, hal ini ditangani melalui mekanisme *look-ahead*. Sebagai contoh, ketika mesin membaca karakter `<`, ia berpindah ke sebuah *intermediate state*. Dari sana, mesin akan "mengintip" karakter berikutnya:

- Jika karakter selanjutnya adalah `=`, mesin bertransisi ke state akhir untuk token `leq` (`<=`).
- Jika karakter selanjutnya adalah `>`, mesin bertransisi ke state akhir untuk token `neq` (`<>`).
- Jika karakter selanjutnya adalah karakter di luar itu, mesin melakukan *retract* (memundurkan pembacaan) dan menerima input sebagai token `lss` (`<`).

2. Pemisahan State Bilangan Bulat (`intcon`) dan Riil (`realcon`)

Penelusuran angka diawali dengan pembacaan digit (`0-9`). Selama input berupa digit, mesin akan terus berputar pada *state* loop angka tersebut. Untuk mengakomodasi konstanta riil (`realcon`), DFA menyediakan jalur transisi khusus jika menemukan karakter titik (`.`). Mesin tidak akan langsung menerima titik tersebut sebagai operator `period`, melainkan menunggu datangnya digit lanjutan. Jika setelah titik dipastikan terdapat digit tambahan, aliran akan bergeser secara definitif ke rute final `realcon`. Jika tidak, mesin akan menangani titik tersebut sebagai leksikal terpisah.

3. Pengelolaan Literal dan Komentar (*Trap States*)

Literal teks dan karakter dipicu oleh tanda petik tunggal (`'`). DFA memiliki *state* khusus yang akan terus melahap semua karakter apa pun hingga menemukan tanda petik penutup. Perbedaan validasi panjang untuk `charcon` (panjang $= 1$) dan `string` (panjang > 1) didelegasikan pada proses operasional program pasca-DFA. Selain itu, DFA juga memetakan rute untuk membuang isi komentar. Pembacaan `{` atau kombinasi `(` diikuti `*` akan membawa mesin ke dalam semacam *trap state* yang terus menerus membaca karakter tanpa membentuk elemen baru, hingga akhirnya menemukan blok penutup `}` atau `*)`.

4. Analisis Penggabungan State untuk Token `ident` dan *Keyword*

Salah satu keputusan desain yang paling krusial pada DFA ini adalah menyatukan penelusuran *Identifier* (`ident`) dan seluruh *Keyword* (seperti `program`, `var`, `begin`, dll) ke dalam satu jalur transisi gabungan yang sama (`q_ident/keyword`). Keputusan arsitektural ini diambil dengan pertimbangan berikut:

- **Kesamaan Leksikal dan Ekspresi Reguler:** Baik *keyword* maupun *identifier* dibentuk oleh pola ekspresi reguler yang ekuivalen sepenuhnya, yaitu `[a-z,A-Z][a-z,A-Z,0-9_]*`. Keduanya wajib diawali oleh alfabet dan bebas diikuti oleh rentetan alfanumerik.

- **Mencegah Fenomena *State Explosion*:** Apabila ke-52 token *keyword* pada spesifikasi Arion dipetakan secara eksplisit huruf demi huruf membentuk sebuah graf berarah (seperti pada struktur data *Trie*), diagram transisi akan menjadi terlampau masif dan tidak proporsional. Hal ini juga memicu kerentanan ambiguitas saat membedakan awalan sebuah variabel (contoh: variabel `integerKu`) dengan *keyword* yang beririsan (`integer`).

- **Efisiensi Resolusi Pascakomputasi:** Melalui penggabungan jalur ini, tugas murni mesin DFA disederhanakan hanya untuk mengekstrak satu rentetan kata yang valid secara leksikal. Setelah *accepting state* tercapai, proses validasi dilimpahkan pada kode program. Algoritma akan melakukan pencarian (*lookup*) nilai *lexeme* tersebut pada Tabel *Keyword* terpusat (*Hash Map*). Jika ditemukan kesamaan, maka token secara spesifik diklasifikasikan sebagai *keyword* (misal: `varsy`). Sebaliknya jika hasil nihil, secara *default* token tersebut diresmikan sebagai `ident` reguler.

3.2 Implementasi Program (Lexical Analyzer)

Tahapan pertama dalam *compiler* ini adalah *Lexical Analyzer* (Scanner) yang bertugas membaca *source code* karakter demi karakter dari `SourceBuffer` dan mengelompokkannya menjadi *Token*. Implementasi pada `Scanner.cpp` dan `Scanner.hpp` dibangun dengan secara langsung mereplikasi logika kerja *Deterministic Finite Automaton* (DFA) yang telah dijabarkan sebelumnya.

3.2.1 Struktur Proyek

Struktur implementasi untuk *lexical analyzer* pada proyek ini berfokus pada folder `src/lexical` dengan pembagian berkas sebagai berikut:

- `SourceBuffer.hpp` & `.cpp`: modul pembaca berkas sumber yang menyediakan akses karakter per karakter secara aman, termasuk operasi utilitas *look-ahead*.

- `Scanner.hpp` & `.cpp`: deklarasi dan implementasi kelas `Scanner` yang menjadi jantung dari mesin *lexical analyzer*.

- `Token.hpp`: representasi unit objek token beserta atribut esensialnya (jenis terminal, nilai *lexeme*, dan metadata penunjuk baris).

- `TokenType.hpp`: definisi enumerasi dari seluruh simpul terminal (52 tipe token) yang dikenali dan dipersyaratkan oleh spesifikasi bahasa Arion.

- `Logger.hpp`: utilitas sistem pelaporan untuk memunculkan pesan peringatan diagnostik selama tahap kompilasi (*lexical errors*).

3.2.2 Klasifikasi Implementasi Alur DFA dalam Program

Siklus mesin direpresentasikan oleh perulangan `scanToken()`. Saat fungsi ini dipanggil, ia bertindak seolah-olah mesin me-reset kembali ke *Start State* (q_0). Percabangan logika implementasi didasarkan pada pembacaan karakter awal `advance()` dengan *switch-case* atau percabangan if, membentuk modul sebagai berikut:

- **Modul 1: Identifier & Keywords (`q_ident/keyword`)**
  Jika evaluasi mengembalikan fungsi valid `isAlpha()`, siklus memanggil `Scanner::identifier()`. Dalam metode ini, program membaca karakter selanjutnya (*looping*) secara kontinu selama fungsi `isAlphaNumeric()` masih bernilai *true*. Saat bertemu karakter asing, iterasi diputus. Teks *lexeme* yang terakumulasi lantas diekstrak dan dicocokkan dengan kontainer tabel konstanta *keyword* (`unordered_map`). Kecocokan berbuah *Token Keyword* (`programsy`, `forsy`, dll); jika tidak, instansiasi mengembalikan *Token* `ident`.

- **Modul 2: Numbers - intcon & realcon (`q_number_int, q_real_number`)**
  Menemukan awalan digit angka mendelegasikan alur ke `Scanner::number()`. Blok perulangan primitif mengekstrak deret murni bilangan bulat. Melalui pendekatan *look-ahead*, mesin kemudian menguji validitas karakter titik '`.`' di depannya. Konformitas *match* (titik diikuti digit) memicu program mengakumulasi komposit tipe nilai primitif bilangan riil (berujung *Token* `realcon`). Tanpa validasi tersebut, nilai default yang diteruskan secara presisi adalah *Token* `intcon`.

- **Modul 3: Strings & Chars (`q_string_char`)**
  Pemicu petik tunggal '`''`' melompat ke rutinitas `Scanner::stringLiteral()`. Secara sederhana program mengolah teks bebas di dalamnya hingga blok penutup kutip terkalibrasi. Selesai memindai, evaluasi dihitung: panjang tepat 1 karakter menyuntikkan kategori `charcon`, sedangkan lebih dari itu bermuara sebagai `string`. Modul ini dibekali dengan interupsi *error handling* seandainya baris berakhir gantung (*Unterminated string*).

- **Modul 4: Operators & Delimiters**
  Karakter tunggal terminal absolut (spt: `+`, `*`, `;`, `,`) memiliki *node accepting state* permanen dan langsung diregister menjadi simbol terminal. Tipe multi-operator memanfaatkan delegasi pergeseran via fungsi `match()`:

  - Simbol `:` akan menguji metode `match('=')`. Retur presisi mencetak `becomes` (`:=`). Ketiadaan retur berbuah terminal tunggal `colon` (`:`).
  - Skema ekuivalen membentengi karakter `<` yang sanggup bercabang ke `leq` (`<=`), `neq` (`<>`), hingga *fallback* tunggal `lss` (`<`). Serta pada `>` yang mencakup `geq` (`>=`) atau `gtr` (`>`).

- **Modul 5: Penanganan Whitespace & Comments**
  Whitespace murni layaknya spasi, tabulasi (`\t`), dan *carriage return* (`\r`) secara utuh digugurkan eksistensinya tanpa mencetak token (lompatan balik siklik pada DFA). Transisi *newline* (`\n`) dimanfaatkan pasif sebagai *tracker* eskalasi baris. Di ranah *comments*, insiden identifikasi pembuka blok (spt `{` atau `(*`) mengaktivasikan pengurasan karakter *dummy loop* (*trap state*) ke dalam *buffer* kosong hingga mendeteksi entitas kutub pasangannya masing-masing.

Seluruh mekanisme program berpedoman pada manuver pergeseran *pointer* alami yang mencerminkan topologi transisi diagram. Kompilasi unit valid tersebut akan diteruskan secara konsisten menuju inisialisasi pada *constructor* statis dari kelas `Token`.

4 Experiments and Results

4.1 Test Case Category A

4.1.1 Test Case 1: *[Description]*

Input: Describe the input used.
Expected output: What the correct result should be.

[Screenshot placeholder]

Gambar 3: Test case 1 — result screenshot

Analysis: Explain whether the result is correct and why.

4.1.2 Test Case 2: *[Description]*

...

4.2 Test Case Category B

...

4.3 Overall Analysis

4.3.1 Algorithm A — Strengths and Limitations

Strengths:

- Strength one.
- Strength two.

Limitations:

- Limitation one.
- Limitation two.

4.3.2 Algorithm B — Strengths and Limitations

...

5 Conclusion

5.1 Summary

1. **Finding 1** — description.
2. **Finding 2** — description.
3. **Finding 3** — description.

5.2 Suggestions for Future Work

1. Idea for improvement or extension one.
2. Idea for improvement or extension two.

5.3 Individual Reflections

5.3.1 Nathanael Imandatua Manurung - 13524021

Role:
Reflection: Write your personal reflection here.

5.3.2 Nicholas Wise Saragih Sumbayak - 13524037

Role:
Reflection: ...

5.3.3 Kurt Mikhael Purba - 13524065

Role:
Reflection: ...

References

[1] A. Author, *Book Title*, Edition. City: Publisher, Year.

[2] A. Author and B. Author, "Article title," *Journal Name*, vol. X, no. Y, pp. 1–10, Month Year. https://doi.org/10.xxxx/xxxxx.

[3] Organisation, "Web resource title," Year. [Online]. Available: https://example.com.

Appendix

Resource	Link
GitHub Repository	https://github.com/your-org/your-repo
Live Deployment	https://your-deployment-url.example.com

Tabel 1: Repository and deployment links